



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

SPRING CLOUD PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on service discovery, gateway, circuit breaking, and tracing

TOTAL: 10 QUESTIONS

Q1 What problems does Spring Cloud solve in a microservices architecture?

Q6 What is distributed tracing in Spring Cloud and how does Micrometer help?

Q2 How does Eureka service discovery work in Spring Cloud?

Q7 How does Spring Cloud LoadBalancer perform client-side load balancing?

Q3 What is Spring Cloud Gateway and how do you define a route?

Q8 What is Spring Cloud Stream and how does it abstract message brokers?

Q4 How does Spring Cloud Config Server manage distributed configuration?

Q9 What is the Saga pattern and how does it apply to Spring Cloud microservices?

Q5 How does Resilience4j circuit breaking work in Spring Cloud?

Q10 How do you propagate OAuth2 tokens between microservices in Spring Cloud?



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Spring Cloud provides turnkey solutions for service

Mitigation

Spring Cloud provides turnkey solutions for service discovery (Eureka), distributed configuration (Config Server), API gateway (Gateway), circuit breaking (Resilience4j), and distributed tracing (Micrometer + Zipkin) the core operational concerns of microservices.

A6 Micrometer Tracing injects a trace ID and

Observability

Micrometer Tracing injects a trace ID and span ID into every request, propagating them via HTTP headers (B3 or W3C). Spring Cloud integrates with Zipkin or Grafana Tempo. All spans for one user request share the same trace ID for correlation.

A2 Services annotated @EnableEurekaClient register with the

Interactives

Services annotated @EnableEurekaClient register with the Eureka server on startup (heartbeat every 30 s). Clients use DiscoveryClient or a load-balanced RestTemplate/WebClient to resolve service names to running instances by querying Eureka.

A7 A @LoadBalanced RestTemplate or WebClient intercepts

Information

A @LoadBalanced RestTemplate or WebClient intercepts requests, resolves the service name via DiscoveryClient (Eureka, Consul), and selects an instance using a RoundRobin strategy. It replaces the deprecated Netflix Ribbon library.

A3 Gateway is a reactive API gateway built

Hardening

Gateway is a reactive API gateway built on Spring WebFlux. Routes are defined by predicates (Path=/api/**) and filters (AddRequestHeader, CircuitBreaker, RateLimiter). Requests matching a predicate are forwarded to the configured URI.

A8 Stream defines Supplier (publisher), Function (processor), and

Performance

Stream defines Supplier (publisher), Function (processor), and Consumer (subscriber) beans that are automatically bound to message broker topics. A binder (Kafka or RabbitMQ) translates between the Spring abstraction and the broker's API.

A4 Config Server serves property files stored in

Gotchas

Config Server serves property files stored in a Git repo (or file system). Client applications fetch their config from Config Server at startup using spring.application.name and active profiles. Refresh without restart is possible via /actuator/refresh + @RefreshScope.

A9 A Saga replaces a distributed ACID transaction

Assessment

A Saga replaces a distributed ACID transaction with a sequence of local transactions, each publishing an event. If a step fails, compensating transactions undo prior steps. Orchestration (a central saga coordinator) or choreography (event-driven) are the two styles.

A5 A circuit breaker wraps a service call

Federation

A circuit breaker wraps a service call and counts failures. When failures exceed a threshold the circuit opens, and calls immediately return a fallback without hitting the downstream service. After a wait duration the circuit enters HALF_OPEN to test recovery.

A10 Configure each microservice as a Resource Server.

Optimization

Configure each microservice as a Resource Server. Use Spring Security's token relay filter or a WebClient filter (ServerBearerExchangeFilterFunction) to forward the incoming Bearer token from upstream to downstream service calls.